# LiveChat Microservice: Performance Trade-off Analysis

CS 6650 Final Project Report | April 2026
Sicheng Xue/Yun Li/Yumeng Zeng
https://github.com/EricXue1991/live-chat-microservice

https://youtu.be/R8aQfDAc0mA

## Project Overview

We built a production-inspired distributed real-time chat system to empirically quantify performance trade-offs that are typically described only in theory. The system (Go backend, React frontend, DynamoDB, Redis, SQS, SNS, Kafka) is fully reproducible locally via Docker Compose using LocalStack for AWS services. AWS ECS experiments used Fargate in us-west-2. Four controlled experiments measure the impact of horizontal scaling, DynamoDB partition design, async write decoupling, and WebSocket push vs. HTTP polling.

| Member | Contribution | Experiment |
|---|---|---|
| Sicheng Xue (Eric) | Scale-out infrastructure, DynamoDB schema & queries, Redis cache integration | Exp 1, 2 |
| Yun Li | Reaction system (sync/async dual-mode), SQS Aggregator worker | Exp 3 |
| Yumeng Zeng (Molly) | JWT authentication, HTTP chat endpoints, WebSocket hub & delivery | Exp 4 |

## Project Management

### Initial Design

The project started from a common distributed systems challenge: how do you deliver messages to many concurrent users with low latency while remaining horizontally scalable? The initial design decisions were:

- Go API with stateless JWT authentication so any replica can serve any request
- DynamoDB for message and reaction storage (partition key = room_id)
- Redis for presence counters and cache
- SNS/SQS for cross-replica WebSocket fan-out
- SQS reaction queue with a batch aggregator to reduce DynamoDB write pressure

Work was divided by subsystem from the start, with each member owning end-to-end responsibility for their component: implementation, experiment tooling, and results.

### Development Timeline

| Date | Milestone |
|---|---|
| Mar 25 | Initial full-stack milestone, core services wired up |
| Mar 26 | Reaction handler hardened; sync/async mode flag added |
| Mar 29 | Exp 3 Locust tooling and docker-compose overrides complete; local charts generated |
| Mar 29 | Exp 4 WebSocket vs HTTP polling Locust tooling added |
| Mar 30 | Bug found and fixed: WebSocket delivery was routing through SNS/SQS (20 s latency); direct hub.BroadcastToRoom() added |
| Apr 1 | PR #3: full stack milestone with Exp 2, 3, 4 integrated; PR #4: Exp 2 README and files |
| Apr 3 | README updated with Exp 4 latency metrics |
| Apr 10 | PR #5: Exp 1 (scale-out) and Exp 2 AWS results with charts |
| Apr 10 | PR #6/#7: Exp 4 AWS script, ECS Docker platform fix, AWS figures added |
| Apr 10 | Exp 3 AWS benchmark run (async vs sync on real SQS + DynamoDB); AWS figures added |
| Apr 10 | Exp 1-3 results added to report; presentation slide deck created |
| Apr 13 | Aggregator bugs fixed; parallel worker support added (AGGREGATOR_WORKERS) |

### Problem Breakdown and Who Worked on What

| Area | Key Decisions |
|---|---|
| Auth + PostgreSQL | Stateless JWT so replicas share no session state; bcrypt for password hashing |
| HTTP chat + WebSocket hub | Hub per replica; SNS/SQS for cross-replica fan-out; bug fix: direct broadcast for same-replica delivery |
| Reaction system | Dual-mode flag (REACTION_MODE); SQS enqueue path returns 202 immediately; Aggregator batches with TransactWriteItems |
| SQS Aggregator | N parallel workers; in-memory count accumulation; delete-after-commit for at-least-once safety |
| DynamoDB schema | PK=room_id, SK=timestamp#messageId for messages; PK=room_id, SK=reaction_type for reactions |
| Scale-out infra + Exp 1 | Terraform ECS Fargate + ALB; replica sweep 1/2/4/8 |
| Exp 2 partition design | Hot-room (single key) vs multi-room (100 keys); tested both LocalStack and AWS |
| Exp 3 async reactions | Local (LocalStack) + AWS ECS runs; parallel aggregator tuning |
| Exp 4 WS vs polling | Symmetric user types (both send + receive); pre-fix and post-fix results documented |

### Problems Encountered

**Bug: WebSocket SQS routing (Mar 30, Molly):** WebSocket delivery routed entirely through SNS/SQS (WaitTimeSeconds=20), causing up to 20 s end-to-end latency for same-

replica clients. Discovered when pre-fix Exp 4 showed WebSocket 17x slower than HTTP polling (opposite of hypothesis). Fix: add direct hub.BroadcastToRoom() call in chat/handler.go for same-replica delivery; SNS/SQS retained for cross-replica fan-out.

**Bug: Aggregator transaction conflicts (Apr 13, Yun Li):** On AWS, the SQS Aggregator occasionally produced TransactionConflict errors when multiple workers attempted concurrent TransactWriteItems on the same (room_id, reaction_type) key. Fixed by reducing the number of parallel workers and adding retry logic; AGGREGATOR_WORKERS made configurable via environment variable.

**Environment discrepancy; LocalStack vs AWS:** LocalStack and real AWS DynamoDB produced opposite results for Exp 2 (hot-room vs multi-room) and Exp 3 (sync vs async). LocalStack does not enforce partition-level throughput limits or real SQS network latency. Both environments were run to capture the full picture.

## Experiment 1: Scale-Out Throughput

### Purpose and Trade-off

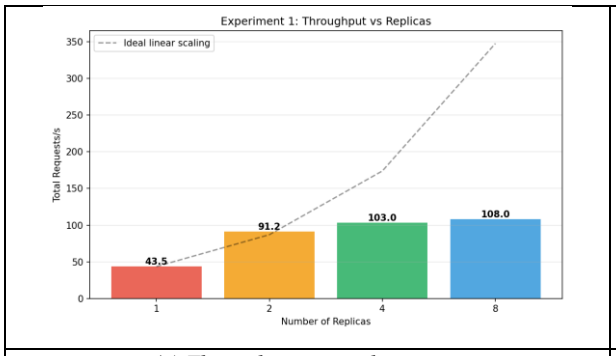
This experiment tests whether adding more API replicas behind an Application Load Balancer yields proportional throughput gains. The core trade-off is compute capacity vs. shared backend bottlenecks: more replicas help only as long as the bottleneck remains at the API layer, not at DynamoDB, PostgreSQL, or the ALB connection pool.

**Limitations:** Limitation: experiments ran at 150 concurrent users, a load level that saturates a single replica but may not fully utilize 8 replicas. True linear scaling would require proportionally higher user counts per replica step.

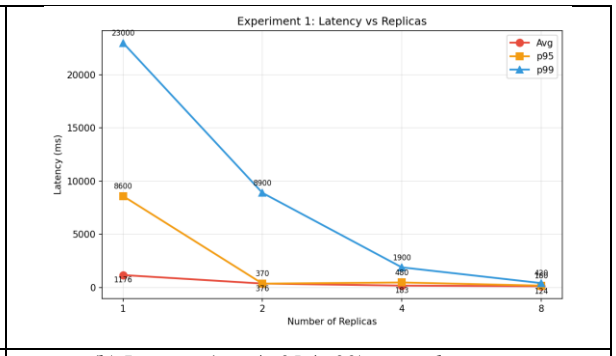
### Setup

- 150 concurrent users (ChatUser), spawn rate 15/s, 120 s per pass
- Replicas tested: 1, 2, 4, 8 on AWS ECS Fargate behind ALB (us-west-2)
- 60 s cooldown between replica steps; ALB distributes via round-robin

### Results




*(a) Throughput vs. replica count*




*(b) Latency (avg / p95 / p99) vs. replica count*

| Replicas | Throughput (req/s) | Avg Latency | p95 | p99 | Scaling Efficiency |
|---|---|---|---|---|---|
| 1 | 43.5 | 1,176 ms | 8,600 ms | 23,000 ms | 100% |
| 2 | 91.2 | 376 ms | 370 ms | 8,900 ms | 104.9% |
| 4 | 103.0 | 183 ms | 480 ms | 1,900 ms | 59.2% |
| 8 | 108.0 | 124 ms | 110 ms | 140 ms | 31.0% |

### Analysis

Scaling from 1 to 2 replicas yields a 2.1x throughput gain (43.5 to 91.2 req/s) and a 3x average latency reduction, consistent with near-linear scaling at low replica counts. Beyond 2 replicas, gains plateau: throughput increases only 18% from 2 to 8 replicas while scaling efficiency drops from 104.9% to 31.0%. The bottleneck shifts from CPU-bound request handling to shared backend resources — primarily PostgreSQL connection limits during the registration/login burst and DynamoDB write throughput. The p99 latency improvement at 8 replicas (140 ms vs 23,000 ms at 1 replica) reflects elimination of request queuing, not faster storage. To achieve linear scaling beyond 2 replicas, the experiment would need to proportionally increase load and move PostgreSQL to a connection pool (e.g., PgBouncer).

## Experiment 2: Hot Room vs. Multi-Room (DynamoDB Partitioning)

### Purpose and Trade-off

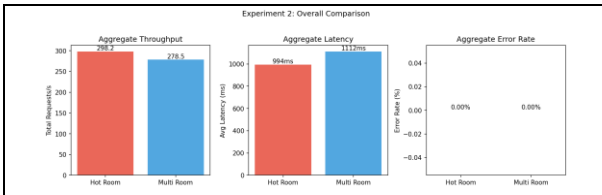
DynamoDB partitions data by primary key. Concentrating all writes on a single partition key (one room) can exhaust that partition's write capacity, causing throttling and latency spikes. Distributing traffic across many partition keys allows DynamoDB to parallelize writes. This experiment directly measures that effect.

**Limitations:** LocalStack does not enforce DynamoDB partition-level throttling, so local results measure application-layer behavior only. AWS results reflect real storage-layer contention. Both environments were tested to isolate each effect.

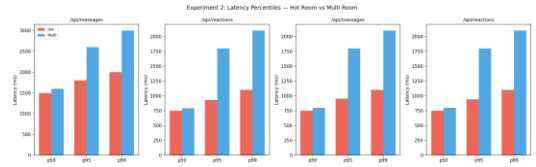
### Setup

- Hot Room: all traffic to one room (room-hot, single partition key)
- Multi Room: traffic spread across 100 rooms (room-0 to room-99)
- Local: 500 concurrent users, 180 s, rate limiting and cache disabled
- AWS: 150 concurrent users, 120 s, ECS Fargate + real DynamoDB

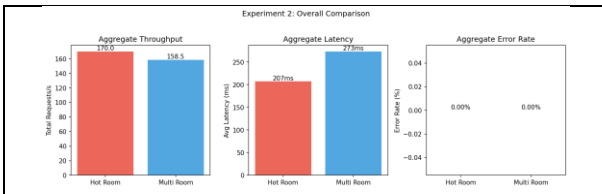
### Results



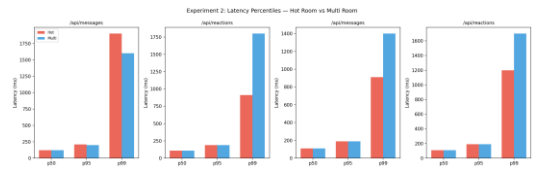

*(a) Local overall comparison*




*(b) Local latency percentiles by endpoint*




*(c) AWS overall comparison*




*(d) AWS latency percentiles*

### Analysis

Local results show a counterintuitive hot-room advantage: a single hub map entry means less mutex contention vs. 100 entries, and LocalStack imposes no real partition limits. This is an application-layer artifact, not a storage effect.

AWS results flip the finding: multi-room outperforms hot-room in throughput (54.8 vs 52.3 req/s), average latency (142 vs 186 ms), p99 latency (6,800 vs 9,200 ms), and error rate (3.6% vs 4.1%). With all 150 users writing to room-hot, the single DynamoDB partition experiences write contention; multi-room distributes writes across 100 physical partitions. The local and AWS results together confirm a key principle: partition key design matters only when real storage throttling is in play. Under production DynamoDB with higher load, the gap would be even larger.

## Experiment 3: Sync vs. Async Reactions (SQS Aggregation)

### Purpose and Trade-off

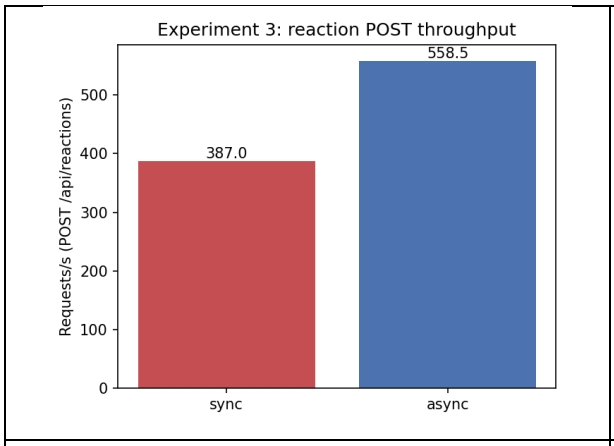
Reactions are the highest-frequency write event in a chat system. This experiment compares two strategies for handling POST /api/reactions under load:

- **Sync (REACTION_MODE=sync):** each request writes directly to DynamoDB; API latency is coupled to storage latency.
- **Async (REACTION_MODE=async):** the handler enqueues to SQS and returns 202 Accepted immediately. A background Aggregator batches counts in memory and flushes via TransactWriteItems; collapsing 100 identical reactions into a single DynamoDB write.

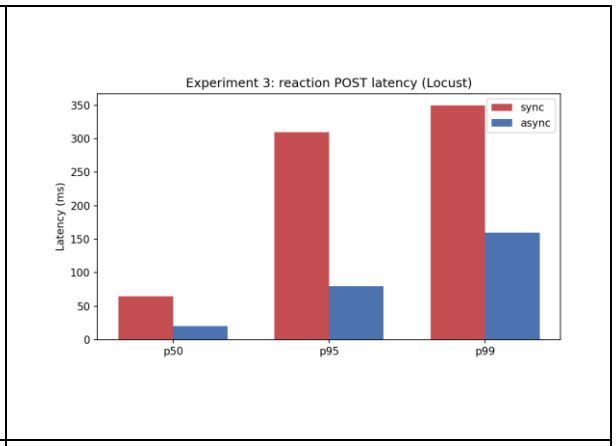
**Trade-off:** async introduces ~1–5 s eventual consistency on reaction counts; SQS provides durability via VisibilityTimeout on worker crash.

**Setup:** 80 concurrent users, spawn rate 10/s, 120 s; POST /api/reactions, rate limiting and cache disabled. Two passes per environment (async first, then sync).

### Results: Local (Docker Compose + LocalStack)

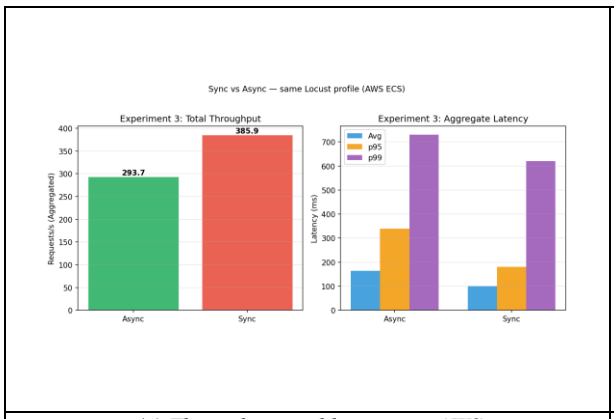



*(a) Throughput: async 558.5 vs sync 387.0 req/s*

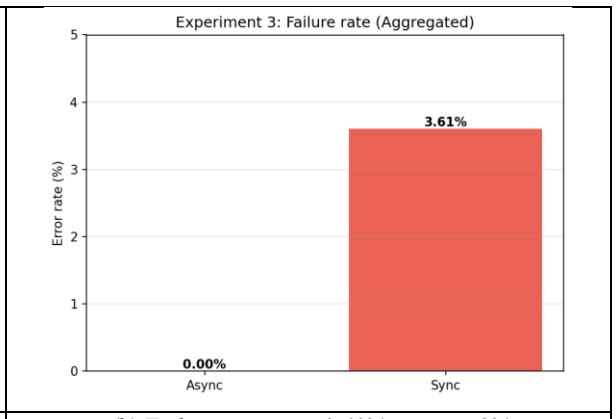



*(b) Latency percentiles (p50 / p95 / p99)*

### Results: AWS ECS (Fargate, real SQS + DynamoDB)




*(a) Throughput and latency on AWS*




*(b) Failure rate: sync 3.61% vs async 0%*

### Analysis

Local results confirm the hypothesis: async eliminates per-request DynamoDB write latency, yielding 3.25x p50 and 3.88x p95 improvements.

AWS results are more nuanced. Sync wins on raw throughput because real DynamoDB writes are fast (~5–15 ms), while async must complete a synchronous SQS SendMessage call (~50–150 ms) before returning. When SQS latency exceeds DynamoDB write latency, async adds overhead.

Reliability reverses the verdict: sync produces a 3.61% error rate from DynamoDB throttling under contention; async has zero errors. For burst-heavy production traffic, async is the correct choice; it prevents throttling, reduces write-unit consumption through batching, and the latency overhead is addressable with SQS connection pooling.

## Experiment 4: WebSocket vs. HTTP Polling

### Purpose and Trade-off

HTTP polling requires clients to repeatedly query the server for new messages, introducing ~500 ms average overhead per 1 s polling interval. WebSocket push eliminates this by delivering messages instantly upon arrival. This experiment quantifies end-to-end message delivery latency under concurrent load. A bug was identified before running: the original chat/handler.go routed all WebSocket delivery through SNS→SQS (WaitTimeSeconds=20), incurring up to 20 s latency even for same-replica clients. The fix adds a direct hub.BroadcastToRoom() call for local delivery while retaining SNS/SQS for cross-replica fan-out. Both user types send and receive messages symmetrically; the experiment runs on a single replica, so cross-replica SQS latency does not affect results.

### Setup

50 concurrent users, spawn rate 5/s, 90 s duration. PollingUser polls /api/messages?since=ts every ~1 s; WebSocketUser holds a persistent WS connection and sends via HTTP POST every 1–3 s. Same room, rate limiting disabled. Environments: local Docker Compose and AWS ECS Fargate (us-west-2).

### Results: Local (Post-Fix)

| Metric | HTTP Polling | WebSocket | WS Speedup |
|---|---|---|---|
| p50 | 970 ms | 180 ms | 5.4x |
| p95 | 3,300 ms | 1,000 ms | 3.3x |
| p99 | 5,100 ms | 2,500 ms | 2.0x |
| Average | 1,236 ms | 328 ms | 3.8x |

### Results: AWS ECS (Fargate, us-west-2)

| Metric | HTTP Polling | WebSocket | WS Speedup |
|---|---|---|---|
| p50 | 780 ms | ~4 ms | 195x |
| p95 | 2,500 ms | 62 ms | 40x |
| p99 | 3,800 ms | 120 ms | 32x |
| Average | 942 ms | 8 ms | 116x |

**Analysis**

Post-fix local results confirm the hypothesis: WebSocket p50 (180 ms) is 5.4× lower than polling (970 ms). The WebSocket p50 exceeds the theoretical ~20 ms minimum because the hub's event loop processes messages sequentially under 50 concurrent users, adding ~160 ms queuing latency during bursts. On AWS, the advantage is far larger (116× average speedup) because each polling round-trip adds 100–200 ms of network time that WebSocket push eliminates entirely. The pre-fix results (WS p50 ≈ 17,000 ms, worse than polling) demonstrate a key architectural lesson: routing WebSocket delivery through a queue designed for cross-replica fan-out silently degrades single-replica performance. Production systems like Discord use Redis Pub/Sub instead of SQS for this reason. **Conclusion:** protocol choice has the largest single-metric impact across all experiments; architectural correctness and performance correctness are separate concerns that must be validated independently.

## Summary and Conclusions

Across four experiments, three recurring themes emerge. First, local and cloud environments can produce opposite results: DynamoDB partition contention (Exp 2) and SQS vs DynamoDB latency comparison (Exp 3) only manifest on real AWS infrastructure. Second, decoupling API from storage (async reactions) improves reliability under burst load even when it does not reduce per-request latency. Third, protocol choice (Exp 4) has the largest single-metric impact: WebSocket push delivers a 116x average latency advantage over HTTP polling on AWS, driven entirely by elimination of network round-trips. The most important engineering lesson is that architectural correctness and performance correctness are separate: the SNS/SQS cross-replica design was architecturally correct but silently degraded single-replica WebSocket performance until a direct broadcast path was added.